

# Streams-and-Tables for Scalable Multi-Camera Object Tracking

Anonymous Author(s)

Affiliation Omitted for Double-Blind Review

**Abstract**—Multi-camera object tracking (MCOT) systems are traditionally constructed as monolithic computer-vision pipelines, causing combinatorial candidate explosions across non-adjacent cameras, burying identity state in program variables, and requiring hard restarts when camera network topology changes. We argue these limitations are architectural rather than algorithmic, and present a relational *streams-and-tables* architecture for MCOT realized on an embeddable Complex Event Processing (CEP) engine: detections flow as append-only streams, identities and topology are queryable relational tables, and cross-camera association is a continuous topology-aware stream join. Evaluated on the Woven WTS and 9-camera NVIDIA SmartSpaces datasets, our system preserves exact functional parity with a JVM port of a state-of-the-art batch pipeline while delivering a  $9.8\times$  speedup ( $12.0\times$  in representative selection,  $9.5\times$  in clustering) and a  $4.16\times$  latency reduction, with live topology events enabling zero-downtime reconfiguration in 190–337 ms.

**Index Terms**—Stream Processing, Complex Event Processing, Multi-Camera Object Tracking, Streams and Tables, Reconfigurable Systems, Edge Computing.

## I. INTRODUCTION

Multi-camera object tracking (MCOT) underpins critical smart city applications, including urban traffic monitoring, intelligent space surveillance, and public safety analytics [1]–[3]. The core task is to assign a unique, continuous global identity ( $ID_G$ ) to each physical object as it traverses a network of cameras  $\mathcal{C} = \{c_1, \dots, c_m\}$ . While single-camera tracking (SCT) within a single field of view (FoV) is well handled by frame-to-frame trackers [1], [4], [5], cross-camera association—matching tracklets across distinct cameras despite lighting changes, viewpoint shifts, occlusions, and non-overlapping spatial gaps—dominates system computational cost and dictates operational scalability.

### A. Problem Statement and Systems Bottlenecks

The computer vision community traditionally treats multi-camera tracking scalability as an *algorithmic* challenge (tighter Re-ID embeddings or heavier graph clustering heuristics). However, deploying MCOT in real-world environments raises three fundamental *systems-level* architectural bottlenecks that academic vision benchmarks largely treat as implementation details:

- 1) **Combinatorial Candidate Explosion:** Naive cross-camera association evaluates all camera pairs, scaling as  $\binom{m}{2} = O(m^2)$ . Real-world networks are spatially sparse; most pairs never observe the same person simultaneously,

yet monolithic pipelines re-evaluate distant pairs indiscriminately.

- 2) **Opaque, Uninspectable State:** Global identity assignments depend on continuously evolving state (tracklet-to-global mappings, last-seen coordinates). In monolithic pipelines this state is buried in imperative variables, making it impossible to query or audit *why* an assignment was made.
- 3) **Brittle, Static Topology:** Camera networks undergo frequent changes: cameras are added, removed, or re-linked. When topology is hardcoded into pipeline logic, reconfiguration requires editing code and restarting the pipeline, destroying accumulated tracking state.

### B. Proposed Paradigm: Streams and Tables

We argue these operational vulnerabilities stem from *monolithic software architecture* rather than vision-algorithm constraints. Drawing on the *stream-table duality* [6], we propose a continuous stream-processing architecture for MCOT. Detections and tracklet updates arrive as append-only event *streams*; global identities and camera topology are maintained as mutable, queryable relational *tables*; and cross-camera association is formulated as continuous spatial joins over live topology relations. Realized on an embeddable CEP engine, high-dimensional feature tensors are shared zero-copy in memory, isolating architectural speedups from runtime/language overheads.

**Contributions:** (1) A streams-and-tables re-architecting of MCOT as an operator graph over detection streams and relational state tables; (2) topology-aware candidate pruning cutting evaluations from  $\binom{9}{2} = 36$  to 6 pairs ( $4.16\times$  latency reduction); (3) five-stage parity verification against a JVM batch baseline on Woven WTS, confirming 100% Global ID parity with a  $9.8\times$  speedup; and (4) an event-driven control plane enabling zero-downtime reconfiguration in 190–337 ms with total state preservation.

## II. BACKGROUND AND RELATED WORK

### A. Multi-Camera People Tracking Baselines

High-performing MCOT solutions catalysed by the AI City Challenge [1]–[3], [7] focus predominantly on computer-vision heuristics. Yoshida et al. [1] proposed Overlap Suppression Clustering (OSC) for offline batch tracking, using degree centrality to resolve overlap anomalies and average-linkage

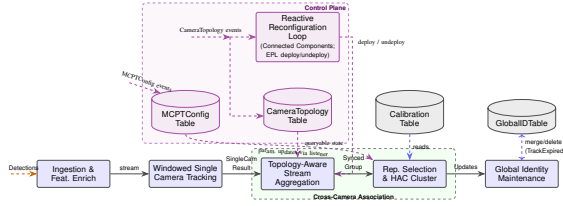


Fig. 1. Proposed operator graph: the *data plane* processes detections through single-camera tracking, topology-aware aggregation, and HAC clustering, with *CalibrationTable* and *GlobalIDTable* as relational state; the *control plane* (violet) upserts *CameraTopology* and *MCPTConfig* events into control tables while a reactive loop deploys/undeploys association queries at runtime with zero downtime.

hierarchical clustering (HAC) over pose keypoint confidence levels. Online entries like Kim et al. [2] (Cluster Self-Refinement) and Xie et al. [3] employ periodic batch clustering windows or geometric projection constraints. However, these systems treat tracking as a monolithic pipeline, relying on periodic offline or semi-batch passes to rectify accumulated errors.

Industrial platforms like NVIDIA DeepStream [8] and Metropolis [9] provide containerized microservice execution, but association logic remains embedded in closed-source modules; state is not natively queryable via SQL, and topology updates require manual service redeployments.

#### B. Stream-Table Duality and CEP Engine Selection

Continuous Query Language (CQL) models [6], [10] unify streaming and relational data across four operator families: stream-to-relation (S2R; temporal windows such as `win:time`), relation-to-relation (R2R; SQL over materialised tables), stream/relation-to-relation (S/R2R; upserts into persistent tables), and relation-to-stream (R2S; timer-driven evictions).

While distributed engines (e.g., Apache Flink [11], Kafka Streams [12]) excel at data-parallel key-value tasks, transferring high-dimensional appearance vectors across network sockets incurs severe RPC serialisation overhead. We select Esper [13], a mature CEP engine providing full CQL semantics in an embeddable, single-process JVM runtime, enabling zero-copy sharing of feature vectors between vision kernels and relational state tables.

### III. SYSTEM ARCHITECTURE

Our system decomposes MCOT execution into two decoupled planes: a *Data Plane* executing continuous tracking operators and a *Control Plane* managing live configuration relations, as illustrated in Fig. 1.

#### A. Data Plane: Incremental Stream Processing

1) **Ingestion and Intra-Camera Overlap Suppression (S2S):** Per-camera detection streams undergo Non-Maximum Suppression (NMS) at ingestion to eliminate overlapping duplicate bounding boxes; this differs from cross-camera field-of-view (FoV) overlap, a spatial relation modeled in *CameraTopologyTable*.



Fig. 2. Topological partitioning of the 9-camera NVIDIA SmartSpaces network into four spatial groups: Group 1 (blue: C02, C17), Group 2 (green: C01, C11), Group 3 (orange: C19, C27), and Group 4 (purple: C21, C25, C30).

2) **Windowed Single-Camera Tracking (S2R):** Local frame detections are grouped into tracklets over a 3-second tumbling window (90 frames at 30 fps, `win:length_batch(90)`), emitting structured single-camera result events.

3) **Representative Feature Selection (S/R2S):** For each tracklet, the system queries *CalibrationTable* to select top- $k$  observations by anatomical keypoint confidence and embedding temporal stability; visibility filters reject occluded boxes before Re-ID matching.

4) **Topology-Aware Aggregation and HAC Clustering (S2R  $\rightarrow$  R2S):** Tracklets in the same spatial neighbourhood are window-synchronized and joined; a windowed Hierarchical Agglomerative Clustering (HAC) kernel then clusters appearance embeddings into global identity candidates.

5) **Global Identity Maintenance (S/R2R and R2S):** Mappings are materialised in a persistent *GlobalIDTable*; cluster events trigger atomic on-merge upserts of last-seen timestamps, while timer queries evict stale tracks (unseen  $> 120$  s) via *TrackExpiredEvent* (R2S) before an R2R deletion garbage-collects heavy feature tensors.

#### B. Cross-Camera Candidate Pruning and Join Planning

Naive all-pairs matching across  $m$  cameras evaluates  $\binom{m}{2}$  pairs. When the network is partitioned into  $k$  spatial topology groups of sizes  $n_1, \dots, n_k$  ( $\sum n_i = m$ ), the required candidate evaluations drop sharply to  $\sum_{i=1}^k \binom{n_i}{2}$ .

For our 9-camera deployment (Fig. 2) partitioned into four spatial groups ( $\{[02, 17], [01, 11], [19, 27], [21, 25, 30]\}$ ), candidate evaluations decrease from  $\binom{9}{2} = 36$  to  $1 + 1 + 1 + 3 = 6$  camera pairs—a **6 $\times$  candidate search space reduction**. Physical execution bakes these topology filters into query plans during compilation, avoiding per-tuple table probes on the hot path.

#### C. Transitional Gating for Non-Overlapping Cameras

To track objects across isolated, non-overlapping cameras without identity fragmentation, the topology relation maintains overlapping edges ( $E_{\text{overlap}}$ ) and directed transition edges ( $E_{\text{transition}}$ ). Global association allows comparing a historical



Fig. 3. Illustrative transition topology mapping for isolated (non-overlapping) cameras. A person leaving the field of view of Camera 12 can transition to Camera 02, Camera 13, or Camera 17. Transitional gating restricts Re-ID candidate matching to these paths, preventing identity fragmentation across non-overlapping zones.

track  $T_H$  (last seen at camera  $c_{last}$ ) with a current cluster  $C_{curr}$  only if  $c_{last}$  belongs to  $C_{curr}$  or shares an overlap/transition edge with some camera in  $C_{curr}$ —restricting matching to physically valid travel routes and preventing false matches across distant areas (Fig. 3).

#### D. Control Plane: Reactive Reconfiguration Loop

Camera topology is stored in a mutable `CameraTopologyTable`. When administrators stream `CameraTopology` update events (e.g., adding a link or camera outage), an on-merge query updates table state while a parallel listener triggers a Java controller (`reconcileDeployments()`) that recomputes connected components, undeploys obsolete aggregation queries, and compiles fresh EPL joins for the new groups—**zero-downtime reconfiguration** without restarts or state loss.

Similarly, tracking parameters (similarity threshold  $simTh$ , clustering radius  $\epsilon_{mcpt}$ ) live in an `MCPTConfigTable` and are re-read on every window call.

### IV. EXPERIMENTAL EVALUATION

#### A. Experimental Setup

All experiments ran on an Intel Core i7-10610U CPU (4 cores, 8 threads, 1.80 GHz), 32 GB RAM, 64-bit Windows 11 Pro, OpenJDK 21 (JBR-21.0.8), with JVM heap capped at `-Xms256m -Xmx256m` to simulate resource-constrained edge gateways and Esper 9.0.0 as the CEP engine. Datasets: (1) Woven VisionAI WTS (Scene 2, 4 cameras) [14] for parity and stage speedups; (2) NVIDIA SmartSpaces (9 cameras) [15] for topological pruning and live reconfiguration.

#### B. RQ1: Five-Stage Implementation Parity and Speedup

To verify that our re-engineering preserves accuracy, we conducted a **five-stage parity evaluation** against a JVM-native batch baseline. As the cited OSC pipeline [1] is Python, we ported it to this functionally equivalent JVM baseline, validated on identical inputs ( $ARI = NMI = 1.0000$ ; similarity-matrix  $MAD = 1.63 \times 10^{-8}$ ). Parity and port-validation metrics are computed by the comparison scripts in `scripts/` over per-window pipeline dumps; since parity holds at every stage,

TABLE I  
FIVE-STAGE PARITY AND ARCHITECTURAL SPEEDUP (WOVEN WTS DATASET)

| Pipeline Stage               | Batch (ms, 25-window total) | Streaming (ms/window, mean $\pm$ std) | Speedup                        | Parity Metric               |
|------------------------------|-----------------------------|---------------------------------------|--------------------------------|-----------------------------|
| Representative Selection     | 10,235.16                   | $34.06 \pm 36.52$                     | <b>12.0<math>\times</math></b> | Keypoint Exact              |
| Clustering (HAC)             | 20,366.55                   | $86.16 \pm 68.86$                     | <b>9.5<math>\times</math></b>  | ARI = NMI = 1.00            |
| Global ID Management         | 521.00                      | $3.13 \pm 1.97$                       | <b>6.7<math>\times</math></b>  | 100% Match                  |
| Matrix Zeroing               | 6.09                        | $0.03 \pm 0.04$                       | <b>8.6<math>\times</math></b>  | $MAD = 0.99 \times 10^{-8}$ |
| Re-ID Similarity Matrix      | 65.08                       | $3.07 \pm 8.28$                       | <b>0.7<math>\times</math></b>  | $MAD = 1.63 \times 10^{-8}$ |
| World Coord. Measure         | 8.65                        | $1.25 \pm 0.79$                       | <b>0.3<math>\times</math></b>  | Exact Match                 |
| World Coord. Replace         | 0.06                        | $0.002 \pm 0.010$                     | <b>1.0<math>\times</math></b>  | Exact Match                 |
| <b>Total MCPT Processing</b> | <b>31,620.00</b>            | <b>129.27 <math>\pm</math> 92.49</b>  | <b>9.8<math>\times</math></b>  | <b>Global ID 100%</b>       |

Note: batch figures are cumulative over 25 windows; streaming figures are per-window means; speedup = batch  $\div$  (streaming  $\times$  25). Sub-1.0 $\times$  stages recompute per window with no amortization benefit; totals include unlisted orchestration overhead.

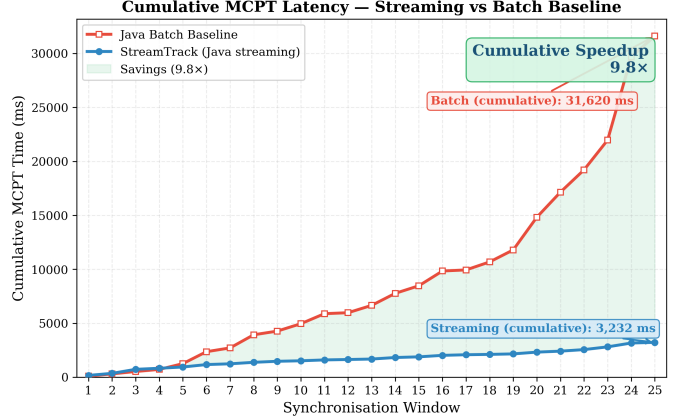


Fig. 4. Cumulative MCPT latency across 25 consecutive windows on the Woven WTS dataset (4 cameras). Incremental streaming achieves a **9.8 $\times$**  speedup over the batch baseline by eliminating redundant state recomputation.

downstream accuracy metrics (IDF1, HOTA) are preserved by construction:

- 1) **Numerical Parity:** Cosine matrices match across RAW, ZEROED, and REPLACED stages ( $MAD = 1.63 \times 10^{-8}$ ).
- 2) **Structural Parity:** Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI) achieve **1.0000** across all 175 commonly-aligned tracklets (14/14 cluster groups match perfectly). The batch pipeline adds one extra representative node (serial 1018)—a pre-clustering artefact with no effect on the shared node set.
- 3) **Post-Processing Parity:** Within-camera NMS and warp-merge fusion generate identical label assignments ( $ARI = NMI = 1.0000$ ).
- 4) **Topology Parity:** The inter-camera association-count matrix converges to an identical  $12 \times 12$  shape (12 representative tracklets in both pipelines).
- 5) **Identity Parity:** Final global person identity assignments match with **100.00% structural equivalence**.

The batch baseline re-reads and re-processes full historical feature archives from disk each window ( $O(N^2)$  growth), requiring 31,620 ms total; our system maintains state incrementally in relational tables, reducing cumulative execution to 3,232 ms—a **9.8 $\times$  end-to-end speedup** (Fig. 4), with **12.0 $\times$**  in representative selection and **9.5 $\times$**  in HAC clustering. The Re-ID similarity matrix and world-coordinate measure stages

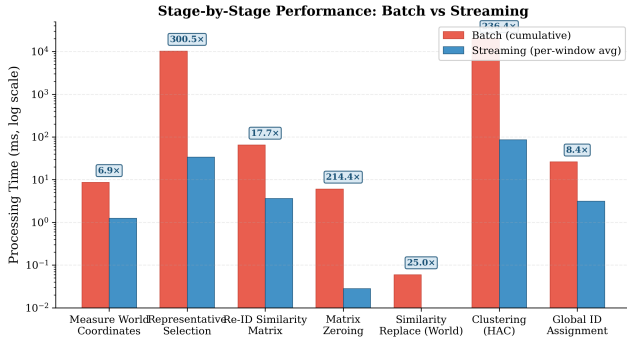


Fig. 5. Stage-by-stage comparison of batch (cumulative over 25 windows) versus incremental streaming (per-window mean). The dominant stages—representative selection and HAC clustering—benefit most from incremental state maintenance (12.0 $\times$  and 9.5 $\times$ ), while per-window recomputation stages show near parity.

TABLE II  
CLUSTERING OPERATOR COMPARISON (NVIDIA SMARTSPACES, 50-WINDOW AVERAGE)

| Algorithm       | Avg. Latency (ms) | ID Precision | Overhead    |
|-----------------|-------------------|--------------|-------------|
| HAC             | 82.4              | 100.0%       | Minimal     |
| CluStream (MOA) | 3,854.2           | 82.5%        | High (Init) |
| ClusTree (MOA)  | 2,510.8           | 78.1%        | Moderate    |

TABLE III  
TOPOLOGICAL OPTIMIZATION IMPACT (NVIDIA SMARTSPACES, 10 RUNS)

| Execution Mode                  | Global (1 Group)     | Topology-Aware (4 Groups) | Speedup       |
|---------------------------------|----------------------|---------------------------|---------------|
| Avg. Latency (inc. JIT warm-up) | 41.96 $\pm$ 22.39 ms | 10.10 $\pm$ 11.11 ms      | 4.16 $\times$ |
| Avg. Latency (exc. JIT warm-up) | 39.04 $\pm$ 18.56 ms | 9.15 $\pm$ 8.27 ms        | 4.27 $\times$ |

show no amortization benefit (0.7 $\times$  and 0.3 $\times$ ): both pipelines recompute these structures per window over the maintained state, whereas the dominant stages (representative selection, HAC clustering) reuse incremental state. Fig. 5 contrasts the per-stage batch-versus-streaming costs directly.

### C. RQ2: Clustering Operator Choice and Topological Optimization

We benchmarked windowed HAC [16] against specialized stream-clustering algorithms (CluStream [17] and ClusTree [18] via MOA [19]). HAC achieves the lowest latency (82.4 ms) and 100% identity precision (fraction of tracklets matching their reference identity) at MCOT window sizes ( $N = 50$ –500), whereas CluStream’s micro-clusters add high initialization overhead (3,854.2 ms; 82.5%) and blur appearance features; ClusTree is comparable (2,510.8 ms; 78.1%), as summarized in Table II.

On the 9-camera NVIDIA SmartSpaces network (Table III), partitioning into 4 topological groups reduces average association latency from 41.96 ms to 10.10 ms (4.16 $\times$  with JVM JIT warm-up) and to 9.15 ms (4.27 $\times$  steady state).

### D. RQ3: Zero-Downtime Runtime Reconfiguration

We validated live resilience by simulating a camera outage: at frame 180, an event disabling Camera 30’s links was injected

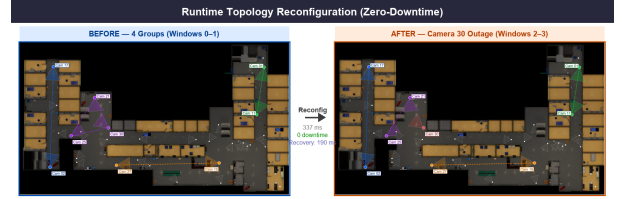


Fig. 6. Runtime topology reconfiguration on the NVIDIA SmartSpaces floor plan. The left panel shows the initial four-group topology; the right panel shows the state after a Camera 30 network outage, where it is isolated as a standalone group (red). The outage transition completed in 337 ms and the subsequent recovery in 190 ms, both with zero downtime.

TABLE IV  
LIVE NETWORK RECONFIGURATION TIMING AND GLOBAL ID STATE CONTINUITY

| Reconfiguration Event       | Deploy/Undeploy  | Time   | Active Global IDs                 |
|-----------------------------|------------------|--------|-----------------------------------|
| Topology Merge (Add link)   | 1 dep., 2 undep. | 181 ms | 1, 2, 3, 4, 5, 6 (Pre-Outage)     |
| Camera Outage (Frame 180)   | 2 dep., 1 undep. | 337 ms | 1, 2, 3, 4, 5, 6 (Outage Win 3)   |
| Camera Recovery (Frame 360) | 1 dep., 2 undep. | 190 ms | 1, 2, 3, 4, 5, 6 (Recovery Win 4) |

Note: parenthetical labels denote the synchronization window at which the active identity set was observed.

without stopping the pipeline; the control loop redeployed affected queries in 337 ms, with recovery at frame 360 in 190 ms—both negligible relative to the 3-second synchronisation window. Fig. 6 visualises the resulting topology transition on the actual floor plan.

All active identities (Global IDs 1–6) persisted across both transitions (Table IV), proving **zero-downtime reconfigurability with 100% state preservation**.

The timing decomposition shows that reconfiguration overhead is dominated by query compilation, not state: the initial deployment of the eight aggregation queries took 1,759 ms (including JVM warm-up and first EPL compilation), while subsequent single-group topology merges completed in 170–191 ms each, all far below the 3-second synchronisation window.

The same event-driven pattern extends beyond topology to the algorithmic parameters of the tracker. `simTh`, `epsilonMcpt`, and `shortTrackTh` are stored in an `MCPTConfigTable` and updated reactively by streaming `MCPTConfig` events; the MCPT kernel re-reads fresh values on every window call. To demonstrate this, we injected three parameter changes at frame 270—tightening the similarity threshold `simTh` from 0.75 to 0.85, reducing the clustering radius `epsilonMcpt` from 0.37 to 0.30, and raising the short-track pruning threshold `shortTrackTh` from 0 to 1—all applied with no restart, no code modification, and no loss of accumulated tracking state, with the new values effective from the very next synchronisation window. This is a capability demonstration of runtime parameter adaptation rather than an accuracy benchmark target; its purpose is to show that the streams-and-tables substrate supports operational tuning without downtime, complementing the latency results measured for topology changes.

## A. Occlusion Handling

Occlusion handling is layered across the pipeline. Intra-camera occlusion is absorbed by the 3-second SCT window: a temporarily occluded person’s tracklet remains active within the window, and visibility filtering rejects heavily occluded boxes before representative selection by requiring a minimum number of high-confidence anatomical keypoints (`keypointConditionTh`). Cross-camera occlusion and temporary disappearance are handled by the `GlobalIDTable`: un-matched identities persist for up to 120 s and re-associate on reappearance through the same topology-gated Re-ID path, while a priority-ordered timer emits `TrackExpiredEvent` to evict only truly stale tracks. The camera-outage experiment of Section IV exercises precisely this mechanism: when Camera 30’s links were disabled, identities 1–6 remained resident in the table across the outage window and re-associated unchanged after recovery—the same state-continuity behaviour required to bridge an occluded interval. Predictive interpolation through occluded zones via CEP patterns is future work.

## B. Operational Advantages

Queryable state lets operators audit identity assignments via standard SQL in real time, and the architecture suggests a natural edge–cloud partitioning. The single-camera tracking stage runs independently per camera and emits one compact `SingleCameraResult` event per synchronisation window—replacing raw video with a low-bandwidth feature stream—while the topology-aware aggregation and global clustering are centralised in the CEP engine. Our measurements support the feasibility of this split: the association-clustering stage processes a 4-camera window in  $129.27 \pm 92.49$  ms on average (streaming) versus 1,265 ms per-window for the batch baseline, and a topology-aware group averages  $9.15 \pm 8.27$  ms in steady state (Table III), so even modest centralised compute can absorb the association workload of many edge cameras.

Two limitations remain: the evaluation is single-node, with physically distributed edge–cloud validation as future work, and topological gating restricts *where* matches are sought but not *how* ambiguous appearances are disambiguated in heavily crowded scenes.

## VI. CONCLUSION AND FUTURE WORK

We presented a streams-and-tables re-architecting of Multi-Camera Object Tracking on an embeddable CEP engine. By converting monolithic pipeline state into queryable relations and cross-camera matching into topology-gated stream joins, it achieves a **9.8×** end-to-end speedup, a **4.16×** topology-aware latency reduction, and sub-second zero-downtime reconfiguration while maintaining 100% functional output parity. Future work includes multi-hop topological joins and physical edge-cloud deployment.

- [1] R. Yoshida, J. Okubo, J. Fujii, M. Amakata, and T. Yamashita, “Overlap suppression clustering for offline multi-camera people tracking,” in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, Seattle, WA, USA, 2024, pp. 7153–7162.
- [2] J. Kim, W. Shin, H. Park, and D. Choi, “Cluster self-refinement for enhanced online multi-camera people tracking,” in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, Seattle, WA, USA, 2024, pp. 7190–7197.
- [3] Z. Xie, Z. Ni, W. Yang, Y. Zhang, Y. Chen, Y. Zhang, and X. Ma, “A robust online multi-camera people tracking system with geometric consistency and state-aware re-id correction,” in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, Seattle, WA, USA, 2024, pp. 7007–7016.
- [4] N. Wojke, A. Bewley, and D. Paulus, “Simple online and realtime tracking with a deep association metric,” in *2017 IEEE International Conference on Image Processing (ICIP)*, 2017, pp. 3645–3649.
- [5] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, “Bytetrack: Multi-object tracking by associating every detection box,” in *Computer Vision – ECCV 2022: 17th European Conference*. Berlin, Heidelberg: Springer-Verlag, 2022, pp. 1–21.
- [6] M. Sax, G. Wang, M. Weidlich, and J. Freytag, “Streams and tables: Two sides of the same coin,” in *BIRTE*. Rio de Janeiro, Brazil: ACM, 2018, pp. 1–10. [Online]. Available: <https://doi.org/10.1145/3242153.3242155>
- [7] A. Specker, “OCMCTrack: Online multi-target multi-camera tracking with corrective matching cascade,” in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, Seattle, WA, USA, 2024, pp. 7236–7244.
- [8] NVIDIA Corporation, “NVIDIA DeepStream SDK – streaming analytics toolkit for AI-based multi-sensor processing,” <https://developer.nvidia.com/deepstream-sdk>, 2024, version 7.x. Accessed: 2026-06-18.
- [9] —, “NVIDIA Metropolis – multi-camera tracking AI workflow,” <https://docs.nvidia.com/metropolis/>, 2025, microservices Reference Architecture 2.1. Accessed: 2026-06-18.
- [10] A. Arasu, S. Babu, and J. Widom, “The CQL continuous query language: semantic foundations and query execution,” *VLDB J.*, vol. 15, no. 2, pp. 121–142, 2006. [Online]. Available: <https://doi.org/10.1007/s00778-004-0147-z>
- [11] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, and K. Tzoumas, “Apache Flink: Stream and batch processing in a single engine,” *IEEE Data Engineering Bulletin*, vol. 38, no. 4, pp. 28–38, 2015.
- [12] J. Kreps, N. Narkhede, and J. Rao, “Kafka: A distributed messaging system for log processing,” in *Proceedings of the Workshop on Networking Meets Databases (NetDB)*, 2011.
- [13] E. Inc., “Esper – complex event processing engine,” <https://www.esper.tech.com/esper/>, 2024, version 9.x. Accessed: 2026-03-13.
- [14] Q. Kong, Y. Kawana, R. Saini, A. Kumar, J. Pan, T. Gu, Y. Ozao, B. Opra, D. C. Anastasiu, Y. Sato, and N. Kobori, “Wts: A pedestrian-centric traffic video dataset for fine-grained spatial-temporal understanding,” 2024, homepage: <https://woven-visionai.github.io/wts-dataset-homepage/>. [Online]. Available: <https://arxiv.org/abs/2407.15350>
- [15] NVIDIA and A. Challenge, “Ai city challenge 2025 track 1: Multi-camera people tracking,” <https://www.aicitychallenge.org/2025-track1/>, 2025, data hosted at <https://huggingface.co/datasets/nvidia/PhysicalAI-SmartSpaces>. Accessed: 2026-03-13.
- [16] D. Müllner, “Modern hierarchical, agglomerative clustering algorithms,” *arXiv preprint arXiv:1109.2378*, 2011.
- [17] C. C. Aggarwal, J. Han, J. Wang, and P. S. Yu, “A framework for clustering evolving data streams,” in *Proceedings of the 29th International Conference on Very Large Data Bases (VLDB)*, ser. VLDB ’03. VLDB Endowment, 2003, pp. 81–92.
- [18] P. Kranen, I. Assent, C. Baldauf, and T. Seidl, “The ClusTree: Indexing micro-clusters for anytime stream mining,” *Knowledge and Information Systems*, vol. 29, no. 2, pp. 249–272, 2011.
- [19] A. Bifet, G. Holmes, R. Kirkby, and B. Pfahringer, “MOA: Massive online analysis,” *Journal of Machine Learning Research*, vol. 11, pp. 1601–1604, 2010.